PM2 & Docker

Why it matters

`node server.js` is how you run an app on your laptop. It is *not* how you run one in production. In production the process must survive crashes (restart automatically), use every CPU core, never lose your one running copy to an SSH disconnect, capture logs somewhere, and roll out new versions without dropping traffic. And it must run *the same way* on a teammate's machine, in CI, and on the server — no "works on my machine." The two tools that dominate this space in the Node world are **PM2** (a process manager) and **Docker** (containerization), and the discipline that ties them together is the **12-factor** methodology.

These come up constantly in interviews because they're where "I can write an endpoint" becomes "I can operate a service." Being able to explain why config belongs in environment variables (not committed files), why you build a Docker image in multiple stages, why the container should run as a non-root user, and how PM2's cluster mode relates to the `cluster` module — that's the operational literacy senior backend roles screen for.

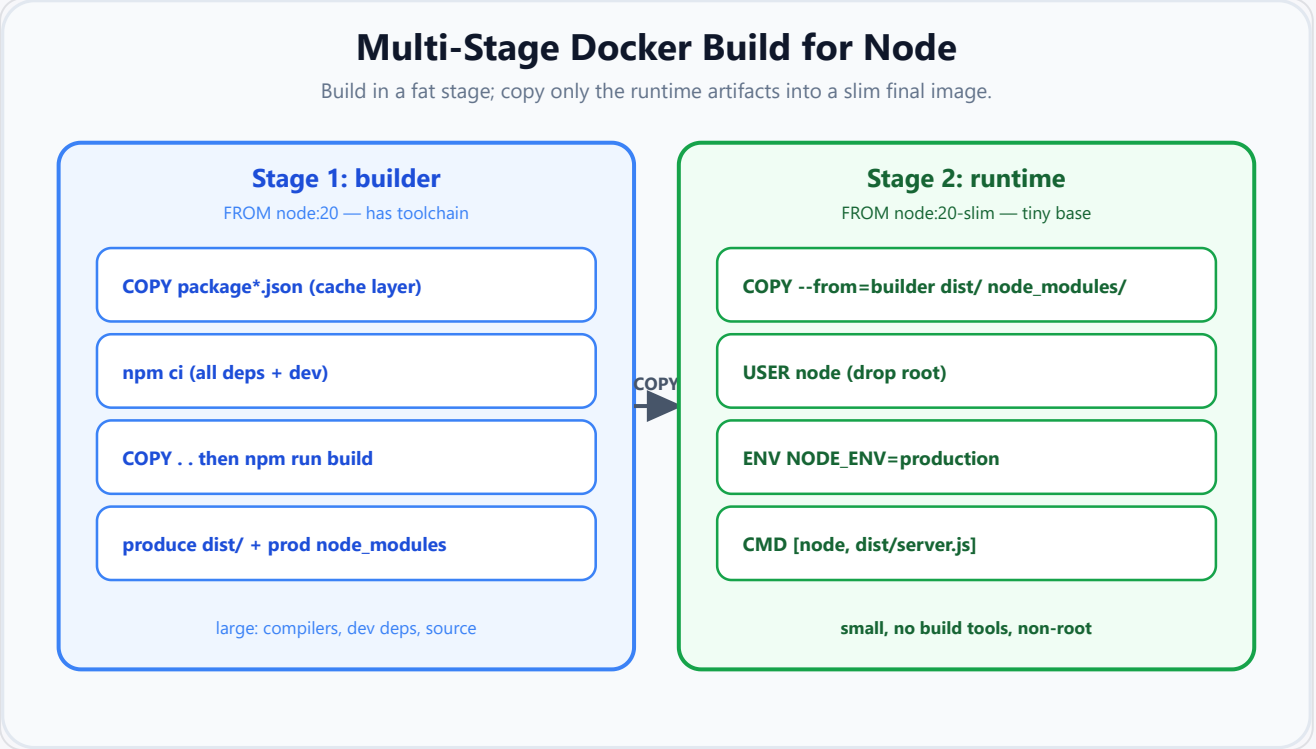
The concept, simply

PM2 is a production process manager for Node. You hand it your app and it keeps it alive: it restarts the process if it crashes or runs out of memory, can run **one instance per CPU core in cluster mode** (wrapping the `cluster` module for you), aggregates logs, and reloads new code with zero downtime. It's the classic way to run Node directly on a VM.

Docker packages your app *and* its entire runtime — the exact Node version, your dependencies, system libraries, everything — into an **image**: a portable, immutable

and makes your app deployable to any container platform (Kubernetes, ECS, Cloud Run).

The key Docker technique for Node is the **multi-stage build**: use a full "builder" image (with compilers and dev dependencies) to install and build, then copy *only* the runtime artifacts into a small, clean final image. The result is a smaller, faster, more secure image with no build tools left inside.



Multi-stage build: a fat builder stage compiles and installs everything; the slim runtime stage copies only the built artifacts and production dependencies, drops to a non-root user, and runs the app.

12-factor is the glue: a set of principles for building services that run well in these environments. The one you'll be asked about most is **config in the environment** — store deploy-specific settings (DB URLs, secrets, feature flags) in environment variables, never hard-coded or committed, so the *same* image runs in dev, staging, and prod with different env values. Other factors that matter here: treat processes as **stateless** (no local session/file state — put it in a backing service), **logs as event streams** to stdout/stderr (let the platform collect them), and **fast startup / graceful shutdown** so instances are disposable.

How it works

exactly like the cluster module (it uses it internally), so you get multi-core scaling and automatic respawn without writing fork/exit logic yourself. An **ecosystem file** (`ecosystem.config.js`) declares apps, instance counts, env vars per environment, memory-restart thresholds, and log paths. `pm2 reload` does a rolling zero-downtime restart (new workers up before old ones retire); `pm2 startup` + `pm2 save` make it survive server reboots.

Docker builds from a `Dockerfile` — an ordered list of instructions, each producing a cached *layer*. Layer caching is why instruction order matters: copy `package.json` and run `npm ci` *before* copying your source, so that changing a source file doesn't invalidate the (slow) dependency-install layer. `npm ci` (not `npm install`) is the production install: it installs exactly what's in the lockfile, deterministically, and is faster in CI. A multi-stage `Dockerfile` has multiple `FROM` lines; the final stage `COPY --from=builder` pulls only what it needs, leaving compilers and dev dependencies behind. A `.dockerignore` keeps `node_modules`, `.git`, and secrets out of the build context.

Security and correctness details the diagram highlights: run as a **non-root user** (`USER node` — the official images ship a `node` user) so a container escape isn't automatically root; set `NODE_ENV=production` (many libraries, including Express, optimize on it, and it prunes dev dependencies); pick a slim base (`node:20-slim` or a distroless/Alpine variant) to shrink attack surface and size; and pin the Node major version rather than `latest`. Handle **PID 1 and signals**: a Node process running as PID 1 in a container doesn't get default signal handlers, so `SIGTERM` may be ignored — use `--init` (or `tini`) and implement graceful shutdown (next topic) so orchestrator stop/rolling-deploy signals are honored.

PM2 vs Docker — do you need both? They overlap on "keep it running and use all cores." A very common modern pattern is **one Node process per container** and let the orchestrator (Kubernetes) handle restarts and horizontal scaling with multiple container replicas — in that world you often *don't* run PM2 cluster mode inside the container, because the platform already supervises and scales. PM2 shines when you deploy straight onto VMs without an orchestrator. Knowing that trade-off is the senior answer.

Code examples

A production multi-stage Dockerfile for a Node app — small, non-root, deterministic.

Node.js Backend Kit

DOCKERFILE

Copy

```
# ——— Stage 1: build ———
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./          # copy manifests first for layer caching
RUN npm ci                     # deterministic install (dev + prod)
COPY . .
RUN npm run build              # produce dist/
RUN npm prune --omit=dev       # drop dev deps for the runtime copy

# ——— Stage 2: runtime ———
FROM node:20-slim AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
USER node                      # drop root privileges
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

A matching `.dockerignore` keeps the build context small and secrets out of the image.

BASH

Copy

```
# .dockerignore
node_modules
npm-debug.log
.git
.env
Dockerfile
dist
coverage
```

Reading config from the environment (12-factor). The same image runs everywhere; only env values change.

Node.js Backend Kit

JAVASCRIPT Copy

```
// config.js — validate env once at startup, fail fast if misconfigured
const required = ['DATABASE_URL', 'REDIS_URL', 'SESSION_SECRET'];
const missing = required.filter((k) => !process.env[k]);
if (missing.length) {
  console.error('Missing env vars:', missing.join(', '));
  process.exit(1);                        // don't boot half-configured
}

export const config = {
  port: Number(process.env.PORT ?? 3000),
  databaseUrl: process.env.DATABASE_URL,
  redisUrl: process.env.REDIS_URL,
  nodeEnv: process.env.NODE_ENV ?? 'development',
};
```

A PM2 ecosystem file for VM deployments — cluster mode, per-env config, auto memory restart.

JAVASCRIPT Copy

```
// ecosystem.config.js
module.exports = {
  apps: [{
    name: 'api',
    script: 'dist/server.js',
    instances: 'max',            // one worker per CPU core (cluster mode)
    exec_mode: 'cluster',
    max_memory_restart: '500M',  // restart a worker if it leaks past 500M
    env: { NODE_ENV: 'development', PORT: 3000 },
    env_production: { NODE_ENV: 'production', PORT: 8080 },
  }],
};
// pm2 start ecosystem.config.js --env production
// pm2 reload api   # zero-downtime rolling restart
```

Common mistakes & gotchas

- **Hard-coding config or committing `.env`.** Secrets in the repo/image leak and can't vary per environment. Read config from env vars; inject secrets at deploy time.
- **Single-stage images with dev tooling inside.** Shipping compilers, dev dependencies, and source bloats the image and widens the attack surface. Use multi-stage and copy only runtime artifacts.
- **Running as root.** The default container user is root; a compromise is then root inside the container. Add `USER node`.
- **Copying source before installing deps.** Breaks layer caching, so every code change reinstalls all dependencies. Copy `package*.json` and `npm ci` first.
- `npm install` **instead of** `npm ci` **in builds.** `install` can drift from the lockfile; `ci` is deterministic and faster for CI/images.
- **Ignoring PID 1 / signals.** Node as PID 1 doesn't get default signal handling, so `docker stop`'s SIGTERM may be ignored and the container is SIGKILLed after a timeout, cutting off in-flight requests. Use `--init`/tini + graceful shutdown.
- **Forgetting** `NODE_ENV=production`**.** Without it you keep dev-only overhead and dev dependencies, and some frameworks run slower.
- **Running PM2 cluster mode *inside* Kubernetes.** Double-scaling (PM2 forks × pod replicas) confuses resource accounting and health checks. Usually one process per container and let the orchestrator scale.
- **Storing state on the container's filesystem.** Containers are ephemeral; local files/sessions vanish on restart. Keep state in backing services (DB, Redis, object storage).
- **Writing logs to files inside the container.** 12-factor says stream logs to stdout/stderr and let the platform collect them.

Best practices

- **Config in the environment**, validated at startup (fail fast on missing vars). One image, many environments.
- **Multi-stage builds**: fat builder, slim runtime; copy only `dist` + prod `node_modules`.

- **Order Dockerfile for cache**: manifests + `npm ci` before source; keep a `.dockerignore`.
- **Handle signals**: `--init`/tini + graceful shutdown so rolling deploys don't drop requests.
- **Stateless processes; logs to stdout**; state and sessions in backing services.
- **Pick one scaling owner**: PM2 cluster on VMs, *or* one process per container + orchestrator replicas — not both.
- **Add a health-check endpoint** and wire it to the platform's liveness/readiness probes.

⭐ Interview lens

Q: Why not just run `node server.js` in production?
A: It doesn't restart on crash, uses only one core, has no zero-downtime reload, and dies with your shell. You need a supervisor — PM2 (restarts, cluster mode, log aggregation, reloads) on VMs, or a container platform (Kubernetes restarts + replicas) that provides the same guarantees.

Q: What is a multi-stage Docker build and why use it for Node?
A: A Dockerfile with multiple stages: a builder stage with the full toolchain installs and compiles, then a slim runtime stage `COPY --from=builder` pulls only the built output and production dependencies. Result: a much smaller image with no compilers, dev deps, or source — faster to ship and a smaller attack surface.

Q: Why put config in environment variables (12-factor)?
A: So the same immutable image runs unchanged in dev, staging, and prod, with only env values differing — and secrets stay out of the repo and image. Hard-

Q: How does PM2 cluster mode relate to the cluster module?
A: PM2 cluster mode uses Node's cluster module under the hood — PM2 acts as the primary and forks one worker per core, sharing the port, so you get multi-core scaling and respawn without writing fork/exit code. It's cluster with production ergonomics (logs, reloads, monitoring).

Q: Why run the container as a non-root user, and what's the PID 1 problem?
A: Non-root (`USER node`) limits blast radius — a container escape isn't automatically host root. PID 1: a process running as PID 1 doesn't get default signal handlers, so `SIGTERM` from `docker stop`/rolling deploys can be ignored and the container gets SIGKILLed, dropping in-flight requests. Fix with `--init`/tini plus graceful shutdown handling.

Q: Do you need PM2 if you're deploying to Kubernetes?
A: Usually not in cluster mode. Kubernetes already restarts crashed containers and scales horizontally with replicas, so running PM2's multi-process cluster inside a pod double-counts CPU/memory and confuses probes. The common pattern is one Node process per container and let the orchestrator own scaling and restarts. PM2 is for VM deployments without an orchestrator.

Cheat-card

- **PM2**: production process manager — restart on crash, `-i max` cluster mode (wraps cluster module), zero-downtime `reload`, boot persistence.
- **Docker image** = app + runtime, portable/immutable; **container** = running instance.
- **Multi-stage build**: fat builder compiles → slim runtime copies only `dist` + prod deps.

- `NODE_ENV=production`, `USER node` (non-root), `--init`/tini for signals.
- **12-factor**: config in env, stateless procs, logs to stdout, fast start / graceful stop.
- Scale owner is **one**: PM2 cluster on VMs *or* one-process-per-container + orchestrator replicas.

QUICK CHECK — no marks, just for you.

1. What is a multi-stage Docker build and why use it for a Node app?

- ◯ A Dockerfile where a fat builder stage installs and compiles, then a slim runtime stage copies only the built artifacts and production deps — giving a smaller, more secure image
- ◯ A build that runs the same image twice to double throughput
- ◯ A build that keeps all compilers and dev dependencies in the final image for debugging
- ◯ A build that splits the app across multiple containers automatically

Check answer

2. Why does the 12-factor methodology say config (DB URLs, secrets) should live in environment variables?

- ◯ Because environment variables are faster to read than a config file
- ◯ Because Docker cannot read files inside a container
- ◯ So the same immutable image runs unchanged across dev, staging, and prod with only env values differing, and secrets stay out of the repo/image
- ◯ Because PM2 requires all configuration to be in the shell environment

3. You deploy to Kubernetes. Should you also run PM2 in cluster mode inside each container?

- ○ Yes, always, so you get double the scaling
- ○ Yes, because Kubernetes cannot restart crashed containers
- ○ It does not matter; the two never interact
- ○ Usually not — Kubernetes already restarts and scales with replicas, so PM2 cluster inside a pod double-counts resources and confuses probes; run one process per container

Check answer

‹ Previous
Caching with Redis

Next ›
Graceful shutdown & monitoring

LEARN MORE

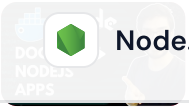
ARTICLES & DOCS

- PM2 Quick Start
  pm2.keymetrics.io
- Containerize a Node.js application (Docker guide)
  docs.docker.com

VIDEOS



How to Use PM2 for Node.js Application Management
Code With KASA



How to Build Node.js Apps with Docker | Dockerize Node.js and Express Apps

Dipesh Malvia

Hand-picked references for this lesson.